

# xv6 ps 시스템 콜 구현

2020038104

김광현

## 1. Design

본 과제에서는 xv6에 새로운 시스템 콜 ps를 추가하여 현재 실행 중인 프로세스의 상태를 출력하는 기능을 구현하였다.

전체 설계는 다음 두 부분으로 구성된다:

1. 유저 레벨 프로그램 (ps.c)
  - 인자를 받지 않으면 전체 프로세스 출력
  - PID를 입력하면 해당 프로세스만 출력
2. 커널 레벨 시스템 콜
  - 프로세스 테이블(proc[])을 순회
  - 각 프로세스의 pid, state, name 출력

이를 통해 간단한 형태의 ps 명령어 기능을 xv6에 구현하였다.

## 2. Implementation

다음과 같은 파일들을 수정 및 추가하였다.

(1) 시스템 콜 번호 추가

kernel/syscall.h

```
#define SYS_ps 22
```

(2) 시스템 콜 선언

kernel/defs.h

```
int ps(int);
```

### (3) 시스템 콜 등록

kernel/syscall.c

```
[SYS_ps] sys_ps,
```

### (4) 시스템 콜 구현

kernel/sysproc.c

```
uint64 sys_ps(void)
```

전달받은 PID 인자를 처리

프로세스 정보를 출력하는 함수 호출

### (5) 프로세스 출력 로직

kernel/proc.c

프로세스 테이블 순회

다음 정보 출력:

PID

상태 (sleep, run 등)

프로세스 이름

### (6) 사용자 프로그램

user/ps.c

```
ps(0);
```

```
ps(atoi(argv[i]));
```

(7) 유저 라이브러리 연결

```
user/user.h
```

```
int ps(int);
```

```
user/usys.pl
```

```
entry("ps");
```

(8) Makefile 수정

```
UPROGS += $U/_ps
```

### 3. Results

(1) 컴파일 과정

```
make clean
```

```
make qemu
```

(2) 실행 결과

```
meta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total
ballocc: first 982 blocks have been allocated
ballocc: write bitmap block at sector 46
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic
legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0
xv6 kernel is booting
hart 1 starting
hart 2 starting
init: starting sh
$ _
```

```
init: starting sh
$ ps
pid,state,name
1,sleep,init
2,sleep,sh
3,run,ps
$ _
```

```
$ ps 1  
  
pid,state,name  
1,sleep,init  
$ _
```

위 결과를 통해 시스템 콜이 정상적으로 동작함을 확인하였다.

#### 4. Troubleshooting

문제 1: undefined reference to 'ps'

원인: 시스템 콜이 제대로 등록되지 않음

해결: syscall.c, sysproc.c 수정

문제 2: 시스템 콜 호출 불가

원인: usys.pl에 entry 누락

해결: entry("ps") 추가

문제 3: 컴파일 에러

원인: 선언 누락

해결: defs.h, user.h에 함수 선언 추가

#### 5. System Call Execution Flow

시스템 콜 ps의 실행 흐름은 다음과 같다:

1. 사용자 입력: ps
2. ps.c에서: ps(0);
3. usys.S를 통해 시스템 콜 호출

4. `ecall`을 통해 커널 진입
5. `syscall()`에서 처리
6. `sys_ps()` 호출
7. 프로세스 테이블 순회 및 출력
8. 사용자 공간으로 복귀